CSI604: OPERATING SYSTEMS

CONTAINER-BASED RESOURCE MANAGEMENT SYSTEM



Lecturer: Dr Diallo Bassoma

GROUP MEMBERS:

1. Kgosi Philip Zilwa-202005452 **(Team Leader)**
2. Phazha Chakalisa-202106101
3. Keletso Joy Ditlhotlhole-202002023
4. Kagiso Mpopang - 202607142
5. Archibald Mareme-201904960

Due Date: 27/10/2026

Table of Contents

1. Introduction 4
   1.1 Problem Statement 4
   1.2 Proposed Solutions 5
      Resource Monitoring 5
      CPU Allocation 5
      Memory Control 6
      System Integration 6
   1.3 Objectives 6
      1.3.1 Primary Objective 6
      1.3.2 Specific Objectives 6
   1.4 Motivation and Project Scope 7
      1.4.1 Motivation 7
      1.4.2 Scope 7
      1.4.3 Deliverables 8
      1.4.4 Exclusions 8
   1.5 Methodology 8
      1.5.1 Development Approach 9
      1.5.2 Development Tools and Resources 9
      1.5.3 Implementation Environment 9
2. Literature Review 9
   2.1 Resource Monitoring 9
   2.2 CPU Allocation 10
   2.3 Memory Control 11
   2.4 Docker as a Virtualization Platform 12
   2.5 Container Management without an Orchestrator 13
   2.6 Related Work Summary 13
3. Requirements Analysis 15
   3.1 Functional Requirements 15
   3.2 Non-Functional Requirements 16
4. System Design 17
   4.1 Architecture 17

4.1.1 Control Loop 18
4.1.2 Resource Policies 19
4.2 Design Decisions 19
4.3 Use Case Model 20
4.4 Behavioural Design 26
Administrator Flows 26
Monitoring Flow 29
Control Flows 29
5. Implementation 33
5.1 Methods 33
Phase 1: Base Environment & Telemetry 33
Phase 2: Dynamic CPU Allocation 33
Phase 3: Memory Safety & Crash Prevention 33
Phase 4: System Integration & Benchmarking 33
5.2 Technologies Used 33
5.3 Implementation Environment 34
6. Experimental Evaluation 35
6.1 Test Environment and Methodology 35
6.1.1 Hardware and Software Configuration 35
6.1.2 Benchmark Tools and Workload Profiles 36
6.1.3 Experimental Methodology 37
6.2 Results 37
6.2.1 Scenario Breakdown and Timeline Analysis 38
6.3 Performance Analysis 41
6.3.1 Comparative Assessment: Static Baseline versus Autonomous Management 41
6.3.2 Evaluation Against Non-Functional Requirements 42
7. Discussion 44
7.1 Strengths 44
7.2 Weaknesses and Limitations 45
7.3 Comparison with Existing Work 45
8. Conclusions and Future Work 46
References 46

1. Introduction

Nowadays, software development relies heavily on microservices deployed in containers using Docker. Containers were established as a mechanism for resource isolation [1], and they keep execution environments lightweight, with low storage overhead and little performance cost. Docker, an open-source platform designed to run applications in isolated containers [2], simplifies deployment by packaging code so that it runs efficiently and consistently from testing through to production. Orchestration platforms such as Kubernetes [3] automate container deployment across clusters, but this project focuses on managing Docker containers directly.

While these platforms make it easier to deploy applications, they introduce a trade-off: the management of how containers share physical CPU and memory resources. When workloads spike unexpectedly, containers compete aggressively for resources, which can cause severe performance degradation, throttling, or a system crash [4].

To address this issue, this project proposes an automated Container-Based Resource Management System. By combining real time performance monitoring with a policy-driven control loop, the system automatically adjusts, throttles or stops containers when workloads spike, without administrator intervention. This approach aims to mitigate out-of-memory (OOM) kills and the noisy neighbour problem and to keep the host stable. This chapter is aimed at introducing the project topic, background information, the problem statement and project objectives. It also covers the approach and methodology by which the project will be carried out, including a breakdown of the project's functionalities to be achieved.

1.1 Problem Statement

The increasing use of containerized applications has made efficient resource management a challenge in modern computing environments. Docker allows multiple applications to run on one host while sharing limited CPU and memory resources. However, high resource consumption by one container can affect the performance of other containers. CPU intensive workloads may reach their allocated limits and experience throttling, while high memory usage may cause containers to exceed their limits and experience Out-Of-Memory (OOM) failures. Although Docker provides CPU and memory limits and soft memory reservations, these remain static unless an administrator changes them, so existing approaches may not always respond

effectively to changing workload conditions. Research done by Lahmann, McCann, and Lloyd found that container memory allocation can differ significantly from actual memory utilization, which can contribute to inefficient resource utilization [5].

Therefore, there is a need to investigate a custom resource management approach that can monitor workloads in real time and make resource allocation decisions based on changing CPU and memory demands. Kim, Lee and Yoo [6] found that containers could experience significant performance degradation even when the resource management mechanisms of an orchestrator (Kubernetes) were used, highlighting the need for improved resource management.

1.2 Proposed Solutions

The proposed solution is to develop a container-based Resource Management System. This project therefore proposes a Container-Based Resource Management System built on Docker, focusing on real time monitoring, CPU allocation and memory control to improve resource utilization and maintain stable container performance. The system operates as a closed loop (measure, evaluate policies, decide, enforce, measure again) and will consist of:

Resource Monitoring

The system will address resource monitoring by using cAdvisor and Prometheus to collect real-time CPU and memory usage, limits and health status from running containers. cAdvisor will gather the resource metrics, whilst Prometheus will store and process the collected data. A dedicated Monitoring Service will ingest this telemetry and serve it to the other services through REST APIs and WebSockets. Resource usage will be compared with each container's assigned limits and with the thresholds defined in the resource policies, so that containers needing resource management can be identified.

CPU Allocation

CPU limits will be configured for running Docker containers to define the maximum CPU resources available to each container. cAdvisor and Prometheus will be used to monitor CPU utilization, allowing containers that stay above a usage threshold for several consecutive intervals to be identified. A Node.js Resource Management Service will evaluate the collected CPU metrics against predefined policies and, when additional processing power is required and available, will issue an increase-CPU decision. An Orchestration Service will apply the decision to the running container through the Docker Engine API, within predefined maximum limits.

Under host-wide memory pressure, the same mechanism will throttle the CPU allocation of non-critical containers.

Memory Control

Memory limits will be configured for running Docker containers to control the maximum amount of memory each container can consume. cAdvisor and Prometheus will continuously monitor memory consumption and provide the metrics needed to identify containers approaching their limits. The Resource Management Service will increase a container's memory limit when its usage stays high for several consecutive intervals, reduce it when usage stays low while leaving headroom, and, if host memory becomes critical, stop the lowest-priority containers so that the host remains stable. This reduces the possibility of Out-Of-Memory (OOM) failures.

System Integration

The resource monitoring, CPU allocation and memory control components will be integrated into a single management system made up of four microservices: a Monitoring Service, a Resource Management Service, an Orchestration Service and a read-only Dashboard UI Service. cAdvisor and Prometheus provide the monitoring data, the Resource Management Service decides when adjustments are required, and the Orchestration Service enforces the decisions on Docker through the Docker Engine API. Policies are stored as data in a validated JSON file and can be updated at run time through a REST API. The backend services will be implemented in Node.js with Express.js.

Finally, the system will be tested using different container workloads and stress tests that generate normal, high and fluctuating CPU and memory usage. Performance will be evaluated using CPU utilization, CPU throttling, memory utilization, resource allocation changes and container stability to determine the effectiveness of the proposed approach.

1.3 Objectives

1.3.1 Primary Objective

To design and develop a Container-Based Resource Management system using Docker to ensure optimal system performance and utilization.

1.3.2 Specific Objectives

- To implement a monitoring mechanism to track real-time container resource usage.

- To construct a CPU allocation mechanism that dynamically manages processing power among running containers.
- To establish memory control rules to set limits and prevent memory exhaustion across the environment.
- To protect the host from memory exhaustion through a host-wide guardrail that freezes scale-ups, throttles CPU and, if necessary, stops low-priority containers.
- To utilize Docker as an open-source platform for virtualization to deliver containers.
- To develop a policy-driven, closed-loop control service in Node.js that automatically evaluates resource policies and enforces the resulting decisions on running Docker containers through the Docker Engine API.

1.4 Motivation and Project Scope

1.4.1 Motivation

As organizations increasingly adopt microservices architectures, managing the computer resources shared among hundreds of containers becomes critical [7]. Without proper governance, a single misconfigured or malfunctioning container can consume disproportionate amounts of CPU or memory (the so-called noisy neighbour problem); this can degrade the performance of the entire system [8]. While Docker offers native resource control mechanisms, administrators often lack a unified, automated, and easily visible way to monitor and enforce these allocations [9]. This project is motivated by the need to optimize and automate infrastructure utilization, prevent system overloads, and ensure predictable performance by building a dedicated management layer for container resources.

1.4.2 Scope

This project focuses on the design, development, and testing of a resource management system integrated with Docker environments. The system's primary boundaries include:

- Resource Monitoring: Real-time tracking and read-only visualization of CPU and memory utilization at the container level.
- CPU Allocation: Automatically increasing or throttling CPU limits for specific containers based on predefined policies.

- Memory Control: Automatically scaling memory limits up or down and applying a host-wide memory guardrail to prevent out-of-memory (OOM) errors and ensure fair memory distribution among active workloads.
- Policy Management and Auditing: Resource policies held as validated data that can be updated at run time, and a timestamped log of every decision with its reason.

1.4.3 Deliverables

- Integrated Resource Management Service: A functional Node.js backend of monitoring, resource management and orchestration services that query cAdvisor/Prometheus metrics, evaluate JSON resource policies and automatically update CPU and memory allocations using the Docker Engine API, together with a read-only dashboard.
- Source Code & Documentation: A structured GitHub repository containing the complete codebase, deployment configuration (Dockerfiles and Docker Compose YAML files), and installation instructions.
- Academic Deliverables: A final CSI604 project report and presentation slides detailing system architecture, methodology, experimental results, and conclusions.

1.4.4 Exclusions

- Legacy Environments: The solution will only support containerized applications running on Docker and will not support traditional Virtual Machines (VMs). It also does not provide orchestration features such as multi-node scheduling, replica scaling or self-healing, which are the role of orchestrators such as Kubernetes.
- Network and Storage Management: The system will not control or monitor network bandwidth allocation or disk I/O throughput.
- Manual Management: The dashboard is read-only. Administrators do not create containers or change limits through it, because resource management is fully automatic.

1.5 Methodology

This chapter will outline the systematic approach, design choices and implementation procedures used to develop the Container based Resource Management System. It establishes a clear step-by-step workflow, taking the project from setting up the environment and collecting data to building the scripts and testing system performance. The full tooling and environment details are consolidated in Chapter 5 of this report.

1.5.1 Development Approach

The project will follow an Agile Iterative Approach, breaking the software project lifecycle into manageable phases so that the resource management modules can be built, tested and refined incrementally. Agile was selected because iterative metric validation allows metric accuracy to be verified in early cycles before complex control logic is written, reaction thresholds and safety margins can be re-tuned after each sprint's performance feedback, and testing components in small increments reduces the risk of undetected bugs cascading between the monitoring layer, CPU controller, and memory safety routines. The work is organized into four phases (base environment and telemetry, dynamic CPU allocation, memory safety and crash prevention, and system integration and benchmarking), which are described in Section 5.1.

1.5.2 Development Tools and Resources

The engineering lifecycle integrates a set of specialized software tools, libraries and utilities spanning telemetry collection, time-series storage, containerization, load generation, API verification and version control. The complete tool list and the role of each tool are presented in Chapter 5, Section 5.2.

1.5.3 Implementation Environment

Development is standardized on Linux platforms to attain real-world system accuracy. The operating system, container runtimes, control scripting language and version-control setup are described in Chapter 5, Section 5.3.

2. Literature Review

This chapter is aimed at discussing various literature on issues on Container based resource management solutions, resource monitoring, CPU allocation and Memory Control as well as existing similar solutions that address the various issues.

2.1 Resource Monitoring

Agarwala et al. [10] note that effective resource monitoring in distributed high-performance environments depends on comprehensive monitoring tools. These monitoring tools are usually utilized to ensure better container resource usage. There are various reasons as to why resource monitoring is performed and one of those being to ensure the availability of the resources [11]. Moreover, there are various mechanisms in place to ensure real time resource monitoring. One of

the mechanisms is building resource monitoring algorithms that ensure that resources are adequately utilized [12]. An example of these algorithms is the "real-time adaptive algorithm" [12]. This algorithm isn't an all size fits all as it was mainly designed for large systems and large data sets. The algorithm uses a three-tier adaptive algorithm that constantly updates how monitoring is done by analyzing the performance of a particular system [12]. In addition, another resource monitoring mechanism is called ReMoS [13]. This system was built for network-sensitive systems that need to be aware of resource availability and was developed to make these systems aware of the network topology using an API.

To address various issues that could come with resource monitoring, container-based solutions were developed. The main motivation was to use various platforms such as Docker and Kubernetes to build containers that will handle resource monitoring. A container-based cluster management system by Google was developed using Kubernetes [14]. That system by Google inspired different solutions such as the Kubernetes cloud resource provisioning tool that was developed to oversee various aspects such as memory, disk and CPU usage [14]. This was achieved by using adaptive monitoring algorithms that reduce data transmission overhead without compromising accuracy [14].

2.2 CPU Allocation

Managing CPU allocation efficiently is crucial to preventing performance bottlenecks in containerized environments. At the operating system level, Batey [15] explains that CPU allocation for Docker containers is fundamentally regulated by Linux control groups (cgroups), specifically using the cpu_quota and cpu_period metrics. These mechanisms dictate the absolute amount of CPU time a container can consume within a predefined timeframe [15]. However, static enforcement of these limits often leads to severe performance issues. For instance, Batey [15] further explains that when a multi-threaded container exhausts its allocated cpu_quota before the end of the cpu_period, the Linux kernel forcibly throttles the container for the remainder of the cycle. This throttling introduces elevated tail latencies and degrades overall application responsiveness, highlighting the inadequacy of static resource limits [15].

To address the limitations of static cgroup allocations, Soth et al. [16] note that container orchestration platforms like Kubernetes offer native dynamic scaling tools, most notably the Horizontal Pod Autoscaler (HPA) and the Vertical Pod Autoscaler (VPA). The HPA adjusts the number of container replicas based on observed CPU utilization, while the VPA dynamically

modifies the container-level CPU requests and limits [16]. While widely adopted, Soth et al. note that these default autoscaling mechanisms are fundamentally reactive; they adjust CPU allocations only after usage thresholds have already been breached. This reactive approach can lead to scaling delays, temporary service-level agreement (SLA) violations, and periods of severe throttling during sudden workload spikes [16].

To achieve truly dynamic and efficient CPU allocation, Mustyala [17] emphasizes shifting from reactive thresholds to proactive, predictive management systems. Studies on dynamic resource allocation highlight that integrating real-time telemetry such as CPU utilization and throttling metrics collected by monitoring tools like Prometheus can be used to anticipate workload demands [17]. Advanced frameworks now explore custom resource controllers and machine learning techniques, such as time-series forecasting, to analyze these metrics and dynamically adjust CPU allocations before a container hits its quota limit [16]. By proactively adjusting processing power based on live performance data rather than waiting for failure conditions, these dynamic systems successfully mitigate resource underutilization, prevent hardware over-provisioning, and maintain application stability [17].

2.3 Memory Control

In modern cloud computing, establishing memory control rules is a fundamental operational requirement to prevent memory exhaustion across containerized environments. Unlike CPU allocations which are compressible and can be throttled during periods of contention, memory is an incompressible resource. Consequently, when a specific container breaches its defined memory limit, the operating system kernel (via Linux control groups or cgroups) typically invokes an Out-Of-Memory (OOM) killer to terminate the container process, preserving global host stability [18]. While Docker (through memory reservations and hard limits) and orchestration platforms such as Kubernetes (through resource "requests" and "limits") allow administrators to establish baseline rules to constrain consumption, configuring these boundaries accurately for complex containerized workloads with unpredictable traffic patterns remains a significant technical challenge [19].

The reliance on static memory rules introduces a critical trade-off between system reliability and cost-efficiency in container clusters. Setting a container's memory limits too conservatively relative to peak application demand frequently triggers disruptive OOM kills, degrading service availability. Conversely, over-provisioning memory limits to safely absorb sudden load spikes or

gradual memory leaks results in severe resource stranding, where paid-for hardware capacity sits idle across the container environment [19]. Studies evaluating traditional orchestration tools, such as Vertical Pod Autoscalers (VPAs), note that these static or highly reactive controllers often struggle to adjust container limits quickly enough in response to sudden memory surges, leading either to immediate OOM errors or delayed, wasteful resource overallocation [18].

To resolve these inefficiencies, recent literature advocates transitioning from static boundaries to dynamic, event-driven, or predictive memory control frameworks designed specifically for containers. Researchers have proposed behavior-driven forecasting models, such as "Theta-Scan," which leverage machine learning to predict imminent memory demand and proactively adjust container limits before exhaustion events occur [19]. Furthermore, advanced event-driven systems like "Escra" operate in sub-second timeframes, capturing near-OOM kernel events to either quickly reclaim memory or dynamically expand a container's allocation right before termination is enforced [20]. By continuously adapting to runtime behaviors, these intelligent systems establish flexible memory control rules that safeguard environment stability while minimizing the financial burden of over-provisioning.

2.4 Docker as a Virtualization Platform

To address the hardware resource overheads associated with traditional hypervisor-based Virtual Machines (VMs), operating system-level virtualization through open-source platforms like Docker has gained widespread adoption. Unlike conventional virtualization, which requires running a complete guest operating system on top of a hypervisor, Docker leverages operating system-level virtualization to execute applications inside isolated execution environments called containers that directly share the host system's kernel. According to Merkel [21], Docker utilizes core Linux kernel primitives, specifically control groups (cgroups) and namespaces, to achieve lightweight virtualization. Namespaces provide process, network, and file system isolation, ensuring that each container operates as an independent unit, while cgroups regulate and limit resource consumption such as CPU cycles and memory usage.

Furthermore, research highlights that Docker delivers significant performance and deployment advantages over traditional virtualized environments. A comparative evaluation by Beserra et al. [22] demonstrated that Docker containers incur negligible execution overhead, consume fewer system resources, and provide near-instantaneous startup times compared to hypervisors, enabling higher container density per host machine. Additionally, Docker standardizes

application delivery through its layered image filesystem architecture. This layered approach allows developers to bundle application code alongside its precise execution dependencies into lightweight, immutable templates, guaranteeing execution consistency across development, testing, and production environments. Consequently, utilizing Docker as an open-source virtualization platform provides the foundation for building dynamic resource allocation tools capable of regulating containerized workloads in real time.

2.5 Container Management without an Orchestrator

Orchestration platforms such as Kubernetes automate the deployment, scaling and recovery of containerized applications, but they add a further layer of configuration and control. This project instead manages containers directly through the Docker Engine, which exposes the same cgroup-based CPU and memory controls on which orchestrators rely (Section 2.4) [21].

Resource contention remains a problem even where an orchestrator is present. Kim, Lee, and Yoo found that containers could experience significant performance degradation even when Kubernetes resource management mechanisms were being used, with CPU contention contributing to performance interference [6]. This supports the need for continuous monitoring and adjustment of container limits, which a Docker-only deployment must provide for itself.

Based on the reviewed literature, this project manages containers directly through Docker and integrates the Docker Engine with cAdvisor, Prometheus and a Node.js management service to monitor resource usage and support dynamic CPU allocation and memory control. Section 7.3 discusses how this choice relates to orchestrator-based approaches.

2.6 Related Work Summary

The reviewed literature establishes several existing approaches to container resource management. Table 2.1 compares the most relevant of these systems against the threshold-based approach adopted in this project.

Table 2.1. Comparison of existing resource management approaches with this project's threshold-based approach

Approach	Mechanism	Response Character	Relation to This Project

Approach	Mechanism	Response Character	Relation to This Project
Kubernetes HPA/VPA [16]	HPA scales replica counts from observed CPU utilization; VPA modifies container-level CPU requests and limits.	Reactive; adjusts only after usage thresholds are breached.	This project also adjusts container-level limits, but through a threshold-and-warning rule set with bounded ceilings, applied directly through the Docker Engine API on a single host without an orchestrator; whether this responds sooner than the VPA is tested in Chapter 6.
Theta-Scan [19]	Behavior-driven machine learning forecasting predicts imminent memory demand and adjusts container limits proactively.	Predictive; acts before exhaustion events occur.	This project uses simpler threshold evaluation; predictive forecasting is identified as future work.
Escra [20]	Event-driven system captures near-OOM kernel events and reclaims or expands memory within sub-second timeframes.	Event-driven, sub-second reaction.	This project relies on periodic Prometheus threshold evaluations rather than kernel event triggers.

Approach	Mechanism	Response Character	Relation to This Project
This project	Closed-loop, policy-driven control: consecutive-breach threshold evaluation over cAdvisor and Prometheus metrics, per-container scale-up and scale-down (Rule A), a host-wide guardrail (Rule B), and bounded CPU and memory adjustment via the Docker Engine API.	Reactive, threshold-based, near real-time.	Baseline approach positioned against the limitations of the systems above.

Across these systems, the consistent lesson is that static limits alone underperform under variable workloads, while the existing dynamic alternatives differ mainly in how early they detect demand: after breach (HPA/VPA), by forecast (Theta-Scan), or by kernel event (Escra). This project deliberately adopts the simplest detectable signal, threshold evaluation over consecutive monitoring intervals, as its control basis.

3. Requirements Analysis

3.1 Functional Requirements

- The system should collect container CPU utilization, memory usage, limits and health status from cAdvisor and Prometheus at a configurable interval (15 seconds by default).
- The system should serve the collected telemetry to the other services through REST APIs and WebSockets.
- The system should evaluate container metrics against resource policies stored in a validated JSON file.
- The system should increase a container's CPU or memory limit, within predefined maximum values, when its usage stays above 90% of its limit for a configured number of consecutive intervals (per-container Rule A, scale-up).

- The system should decrease a container's memory limit, leaving 25% headroom, when its usage stays below 35% of its limit for a configured number of consecutive intervals (Rule A, scale-down).
- The system should apply a cooldown period after every limit change to prevent flapping.
- When the total memory used by all containers exceeds 98% of host RAM, the system should freeze new scale-ups, throttle the CPU of non-critical containers and, if host memory remains critical, terminate the lowest-priority containers one by one (host-wide Rule B, which always overrides Rule A).
- The system should resume normal operation once host memory drops below 90%, and should never throttle or evict protected workloads such as prometheus, backend and dashboard.
- The system should separate decision-making (Resource Management Service) from enforcement (Orchestration Service), with the Orchestration Service applying decisions to containers through the Docker Engine API.
- The system should skip a control cycle when the Monitoring Service is not healthy, so that no decision is based on stale or incomplete data.
- The system should allow administrators to update resource policies at run time through a REST API, without restarting any service.
- The system should record every decision with a timestamp and a reason in an auditable log.
- The system should provide a read-only dashboard that shows metrics, alerts and system status, and that cannot create containers or change limits.

3.2 Non-Functional Requirements

- The monitoring and management components should consume no more than 5% of host CPU on average during normal operation, to minimize the overhead introduced by resource monitoring and management.
- The system should refresh and process CPU and memory usage data at an interval of 15 seconds or less to provide near real-time monitoring.
- The system should be able to use portable configuration files, such as YAML or Docker Compose, for container, monitoring and resource configurations.
- The system should monitor at least 20 containers simultaneously on the test host without the refresh interval exceeding the 15-second target.

- The system should keep workload containers running when Rule A adjusts their limits, with no container restarts or OOM kills caused by a scale-up or scale-down; containers are terminated only by the host guardrail (Rule B) and never if they are protected.
- The host guardrail should bring host memory back below the 90% recovery threshold within 15 seconds of activation.
- The system should be able to prevent resource allocations from exceeding predefined CPU and memory limits.
- The system should recover from temporary failures of the monitoring or resource management components within 60 seconds and without manual intervention.
- The system should be able to secure access to the Docker Engine API (for example, by not exposing it over an unauthenticated network socket) and restrict unauthorized resource-management operations.
- The system should be able to record monitoring data, every resource decision with its reason, warnings, and errors.
- The system should validate policy updates before accepting them, so that an invalid policy is rejected and the previous policy remains active.
- The system should be able to process resource usage data and make allocation decisions without introducing significant overhead to running containers.

4. System Design

4.1 Architecture

The system follows a microservice architecture consisting of four independent, cooperating services, which separates concerns, eases testing and lets each service be developed on its own. The Monitoring Service ingests live telemetry (CPU, memory usage, limits and health status) from cAdvisor, which is deployed alongside the workload containers, and from Prometheus, which stores it as time-series data, and serves it to the other services through REST APIs and WebSockets. The Resource Management Service is the decision-maker: it periodically fetches metrics from the Monitoring Service, evaluates them against resource policies held in a validated JSON file, and produces decisions (SET_MEMORY, INCREASE_CPU, THROTTLE_CPU, TERMINATE and FREEZE). The Orchestration Service is the enforcer: it connects to the Docker Engine API, receives or polls the decisions, and applies them by changing the limits of

running containers or by stopping containers. The Dashboard UI Service is a read-only interface through which administrators view metrics, alerts and system status; it cannot create containers or change limits, because management is fully automatic.

The overall data flow proceeds from the Docker Engine through cAdvisor to Prometheus and the Monitoring Service, then to policy evaluation in the Resource Management Service, and onward through the Orchestration Service and the Docker Engine API to the running containers. The dashboard reads its displayed data from the Monitoring Service.

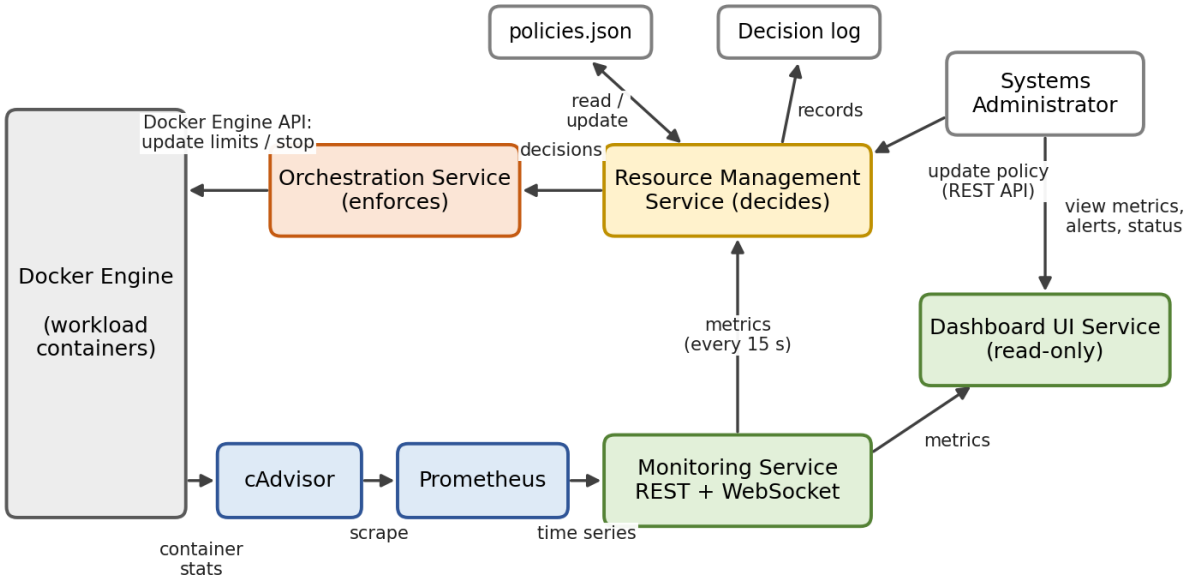


Figure 4.1. System architecture of the Container-Based Resource Management System

4.1.1 Control Loop

The core of the system runs continuously on a configured scrape interval (15 seconds by default): measure, evaluate policies, decide, enforce, and measure again. Each cycle has five steps.

- Health check: the Resource Management Service checks that the Monitoring Service is up. If it is degraded, the cycle is skipped to prevent bad decisions.
- Metrics fetch: it retrieves live CPU and memory statistics for all containers.
- Policy evaluation: the metrics are evaluated against the configured policies, Rule B first and then Rule A.

- Decision dispatch: decisions are pushed to, or polled by, the Orchestration Service.
- Enforcement: the Orchestration Service applies the new limits, throttles CPU or terminates containers as instructed.

4.1.2 Resource Policies

Policies are stored as data in a validated policies.json file and can be updated at run time through a REST API without restarting services or changing code. They are split into two rules.

- Rule A, per-container thresholds (scale-up): if a container's memory or CPU stays above 90% of its assigned limit for N consecutive intervals (for example 3 intervals, about 45 seconds), its limit is increased, for example by 256 MiB of RAM or 0.25 cores.
- Rule A, per-container thresholds (scale-down): if a container's memory stays below 35% for 8 consecutive intervals, its limit is decreased, leaving 25% headroom. A cooldown timer (for example 60 seconds) follows every change to prevent flapping.
- Rule B, host-wide guardrail: it activates when the total memory used by all containers exceeds 98% of host RAM and always overrides Rule A. Stage 1 (freeze) blocks new scale-up requests; stage 2 (throttle) reduces the CPU allocation of all non-critical containers to a minimum baseline; stage 3 (evict) terminates the lowest-priority containers one by one while host memory remains critical.
- Recovery and protection: once host memory drops below 90%, the freeze is lifted and normal operation resumes. Critical containers (for example prometheus, backend and dashboard) are protected and are never throttled or evicted.

4.2 Design Decisions

Ten key design decisions shape the system:

- Microservices over a monolith: the monitoring pipeline and control logic have different load profiles, so separating them keeps metric collection overhead independent of control complexity, consistent with the requirement that monitoring run with minimal overhead.
- Threshold-based dynamic adjustment instead of static limits or an orchestrator's built-in autoscalers: literature indicates that static limits and default orchestrator mechanisms may not respond adequately to changing workloads [6], [16]. Consecutive-breach detection prevents overreaction to split-second spikes, and scaling up once usage stays above 90% of a limit

matches the goal of expanding headroom before OOM failures; the resulting responsiveness is measured in Chapter 6 rather than assumed.

- Monitoring Service as the single source of telemetry: all other services read container metrics from the Monitoring Service, which is fed by cAdvisor and Prometheus, rather than querying containers directly, isolating the system from container failures and matching the recovery requirement.
- Two-rule policy with a host guardrail: per-container thresholds (Rule A) keep individual containers healthy, while the host-wide aggregate rule (Rule B) overrides Rule A to protect the machine; protected workloads such as prometheus, backend and dashboard are never throttled or evicted.
- Cgroups counters as the measurement basis: decisions are computed from kernel counters so telemetry reflects real resource isolation semantics [15].
- Run-time limit updates through the Docker Engine API: CPU and memory limits are changed on running containers with the Docker update operation (the equivalent of docker update), which modifies a container's cgroup limits without recreating it, so the workload keeps running.
- Docker without an orchestrator: the system targets single-host deployments, so adding a cluster layer would add complexity without benefit; orchestration features such as multi-node scheduling and self-healing are outside the project's scope.
- Separation of deciding and enforcing: the Resource Management Service only decides and the Orchestration Service only enforces, so policy logic can be tested without touching Docker.
- Policies as data: rules are held in a validated JSON file and can be updated at run time through a REST API without restarting services or touching the codebase.
- Safe by design: consecutive-breach detection, cooldown timers after every change and a wide gap between the scale-up (90%) and scale-down (35%) thresholds prevent flapping, and every decision is logged with its reason.

4.3 Use Case Model

Figure 4.2 shows the use case diagram. The Systems Administrator is the only human actor and interacts with three use cases: View Metrics, View Alerts and Decision Log, and Update Resource Policy (through the REST API rather than the dashboard, which is read-only). The

remaining use cases are automatic and are driven by cAdvisor and Prometheus, the Resource Management Service and the Orchestration Service: Collect Metrics, Evaluate Container Policy, Apply Host Guardrail, Enforce Decision, and Record Decision and Raise Alert. View Metrics includes Collect Metrics, Evaluate Container Policy includes Enforce Decision, and Enforce Decision includes Record Decision and Raise Alert. Apply Host Guardrail extends Evaluate Container Policy, because Rule B overrides Rule A whenever host memory is critical.

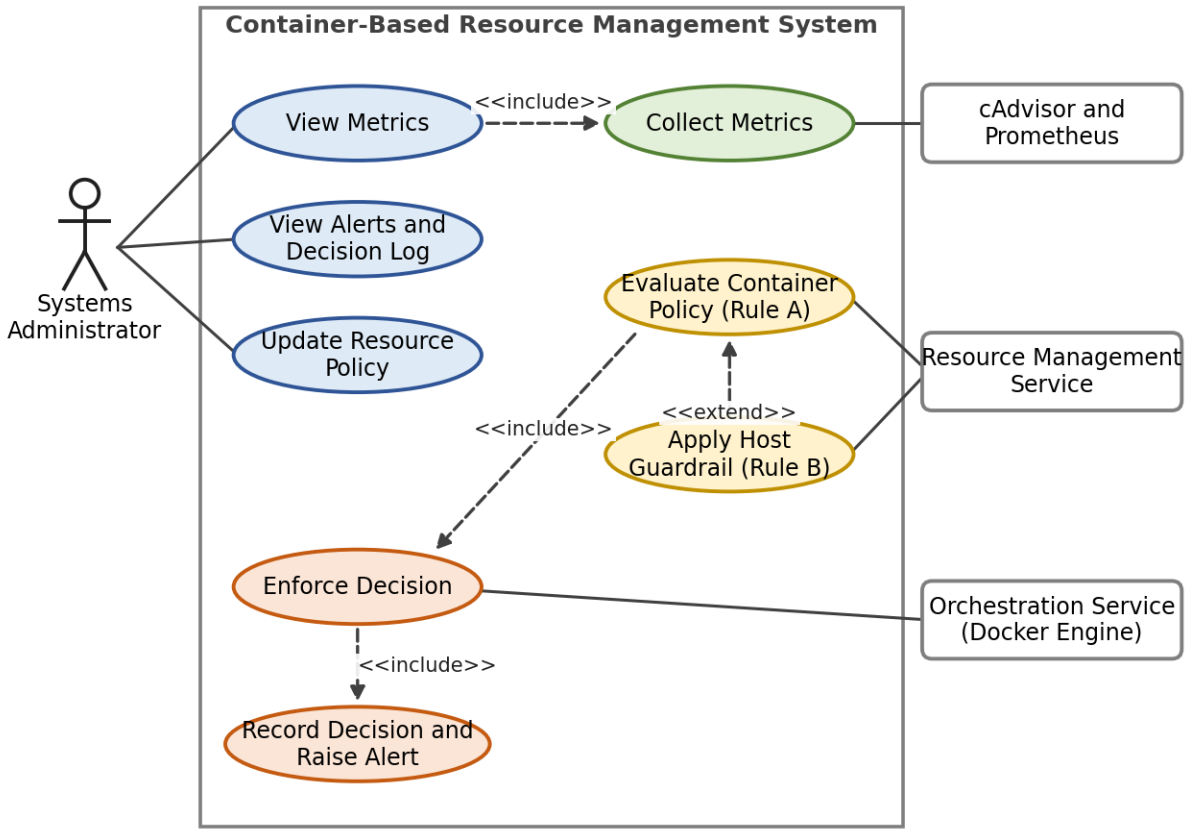


Figure 4.2. Use case diagram of the Container-Based Resource Management System

Table 4.1. Use Case 1: View Metrics

Name	View Metrics
Participating Actors	Systems Administrator
Entry Conditions	The use case starts when the Systems Administrator opens the read-only dashboard.

Flow Of Events	1. The Systems Administrator opens the dashboard. 2. The Dashboard UI Service requests the latest metrics and system status from the Monitoring Service and subscribes to live updates. 3. The system includes Collect Metrics so that the displayed data is current. 4. The dashboard displays CPU usage, memory usage, limits and health status for each container.
Exit Conditions	The use case ends when the Systems Administrator closes the dashboard.
Exceptions	The Monitoring Service is unavailable and the dashboard shows a notice that metrics cannot be retrieved.
Special Requirements	The dashboard is read-only: it offers no controls for creating containers or changing limits. Data refreshes automatically at the scrape interval.

Table 4.2. Use Case 2: View Alerts and Decision Log

Name	View Alerts and Decision Log
Participating Actors	Systems Administrator
Entry Conditions	The use case starts when the Systems Administrator opens the alerts and decision log view.
Flow Of Events	1. The Systems Administrator opens the alerts and decision log view. 2. The Dashboard UI Service retrieves recent alerts and logged decisions. 3. The dashboard lists each entry with its timestamp, container, action and reason.
Exit Conditions	The use case ends when the Systems Administrator closes the view.
Exceptions	The decision log cannot be read and the dashboard shows a notice that the log is unavailable.
Special Requirements	The view is read-only. Every decision shown carries a timestamp and a justification.

Table 4.3. Use Case 3: Update Resource Policy

Name	Update Resource Policy
Participating Actors	Systems Administrator
Entry Conditions	The use case starts when the Systems Administrator submits a new or modified policy to the policy REST API.
Flow Of Events	1. The Systems Administrator submits the policy as JSON through the REST API. 2. The system validates the policy structure and values (thresholds, intervals, cooldown, protected workloads). 3. If the policy is valid, the system replaces the active policy at run time. 4. The system confirms that the new policy is active and records the change.
Exit Conditions	The use case ends when the new policy is active and the Systems Administrator has received a confirmation.
Exceptions	The policy is invalid: it is rejected, an error message is returned and the previous policy remains active.
Special Requirements	No restart of any service and no code change is required. Policies are changed through the REST API, not the dashboard.

Table 4.4. Use Case 4: Collect Metrics

Name	Collect Metrics
Participating Actors	cAdvisor, Prometheus
Entry Conditions	The use case starts when the configured scrape interval elapses (15 seconds by default).
Flow Of Events	1. cAdvisor reads the CPU, memory and limit statistics of each container from the Docker Engine. 2. Prometheus scrapes the statistics from cAdvisor and stores them as time series. 3. The Monitoring Service queries Prometheus and exposes the metrics through its REST and WebSocket interfaces.

Exit Conditions	The use case ends when the latest metrics are available to the other services.
Exceptions	A target container or cAdvisor cannot be reached: the failure is logged and collection is retried at the next interval.
Special Requirements	Collection must add minimal overhead and complete within the scrape interval.

Table 4.5. Use Case 5: Evaluate Container Policy

Name	Evaluate Container Policy
Participating Actors	Resource Management Service
Entry Conditions	The use case starts when a control cycle begins and the Monitoring Service reports UP.
Flow Of Events	1. The service checks the health of the Monitoring Service and skips the cycle if it is degraded. 2. The service fetches the CPU and memory metrics of all containers. 3. For each container, it compares usage with the assigned limit and counts consecutive intervals above 90% or below 35%. 4. When the required count is reached and the container is not in cooldown, the service produces a SET_MEMORY or INCREASE_CPU decision within the predefined maximum limits. 5. The service includes Enforce Decision to apply the decision.
Exit Conditions	The use case ends when all decisions for the cycle have been dispatched.
Exceptions	The Monitoring Service is degraded and the cycle is skipped. A container is in cooldown and no decision is made. A host-wide freeze is active and scale-ups are blocked.
Special Requirements	A scale-up adds a fixed step (for example 256 MiB of RAM or 0.25 cores); a scale-down leaves 25% headroom; a cooldown (for example 60 seconds) follows each change.

Table 4.6. Use Case 6: Apply Host Guardrail

Name	Apply Host Guardrail

Participating Actors	Resource Management Service
Entry Conditions	The use case starts when the total memory used by all containers exceeds 98% of host RAM.
Flow Of Events	1. The service freezes all new scale-up requests from Rule A. 2. The service reduces the CPU allocation of all non-critical containers to a minimum baseline (THROTTLE_CPU). 3. If host memory remains critical, the service terminates the lowest-priority containers one by one (TERMINATE). 4. When host memory drops below 90%, the service lifts the freeze and normal operation resumes.
Exit Conditions	The use case ends when the freeze is lifted.
Exceptions	Only protected containers remain and no further container can be stopped: an alert is raised.
Special Requirements	Rule B always overrides Rule A. Protected workloads (for example prometheus, backend and dashboard) are never throttled or evicted.

Table 4.7. Use Case 7: Enforce Decision

Name	Enforce Decision
Participating Actors	Orchestration Service, Docker Engine
Entry Conditions	The use case starts when a decision is pushed to, or polled by, the Orchestration Service.
Flow Of Events	1. The Orchestration Service receives the decision. 2. The service checks that the target container exists and, for throttling or termination, that it is not protected. 3. The service applies the decision through the Docker Engine API by updating the container limits or stopping the container. 4. The service reports the outcome to the Resource Management Service and includes Record Decision and Raise Alert.
Exit Conditions	The use case ends when the decision has been applied and its outcome reported.

Exceptions	The Docker Engine API call fails: the failure is recorded and an alert is raised. The host lacks the free memory or CPU for a requested increase and the decision is rejected.
Special Requirements	The Orchestration Service only enforces decisions; it never decides.

Table 4.8. Use Case 8: Record Decision and Raise Alert

Name	Record Decision and Raise Alert
Participating Actors	Resource Management Service, Orchestration Service
Entry Conditions	The use case starts when a decision is made or enforced, or when a threshold or guardrail event occurs.
Flow Of Events	1. The system writes a log entry with the timestamp, container, action and reason. 2. For warnings, freezes, throttling, evictions and failed enforcement, the system builds an alert. 3. The system makes the alert and the log entry available to the Dashboard UI Service.
Exit Conditions	The use case ends when the entry is stored and visible on the dashboard.
Exceptions	The log cannot be written: the error is reported on the service console and the control cycle continues.
Special Requirements	Every decision must be auditable, with a timestamp and a stated reason.

4.4 Behavioural Design

The dynamic behaviour of the system is specified with one sequence diagram per use case. The diagrams are grouped into administrator flows, the monitoring flow and the control flows, and follow the order of the use case tables.

Administrator Flows

In the View Metrics sequence (Figure 4.3), the Systems Administrator opens the read-only dashboard. The Dashboard UI Service requests the latest metrics and status from the Monitoring Service, subscribes to live updates over a WebSocket and displays the CPU usage, memory

usage, limits and health status of every container. If the Monitoring Service is unavailable, the dashboard shows an error notice instead.

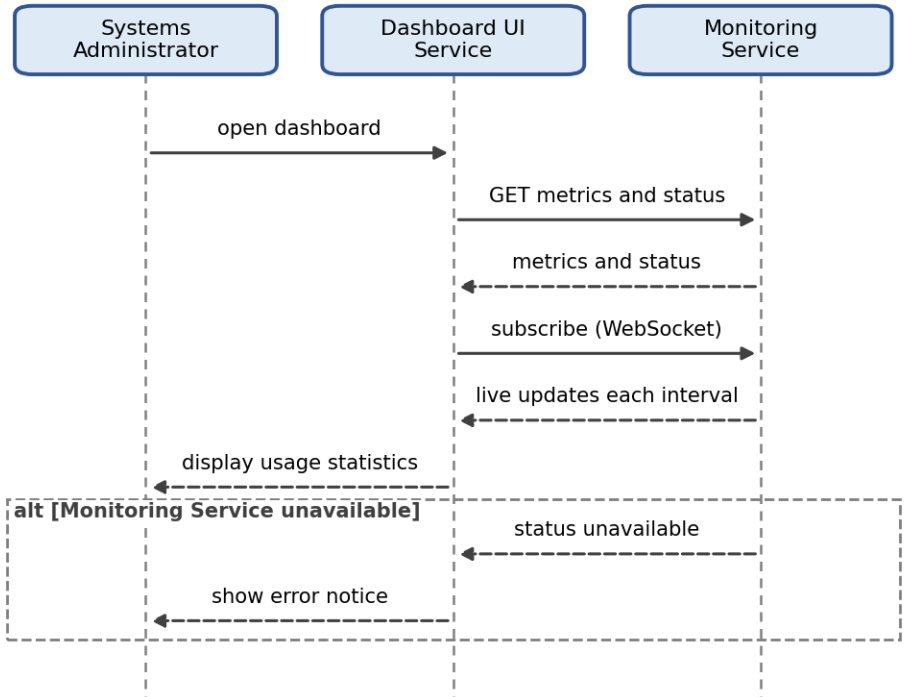


Figure 4.3. View Metrics sequence diagram

In the View Alerts and Decision Log sequence (Figure 4.4), the Systems Administrator opens the alerts view. The Dashboard UI Service requests the recent alerts and decisions from the Resource Management Service, which reads them from the decision log, and the dashboard lists each entry with its timestamp, container, action and reason.

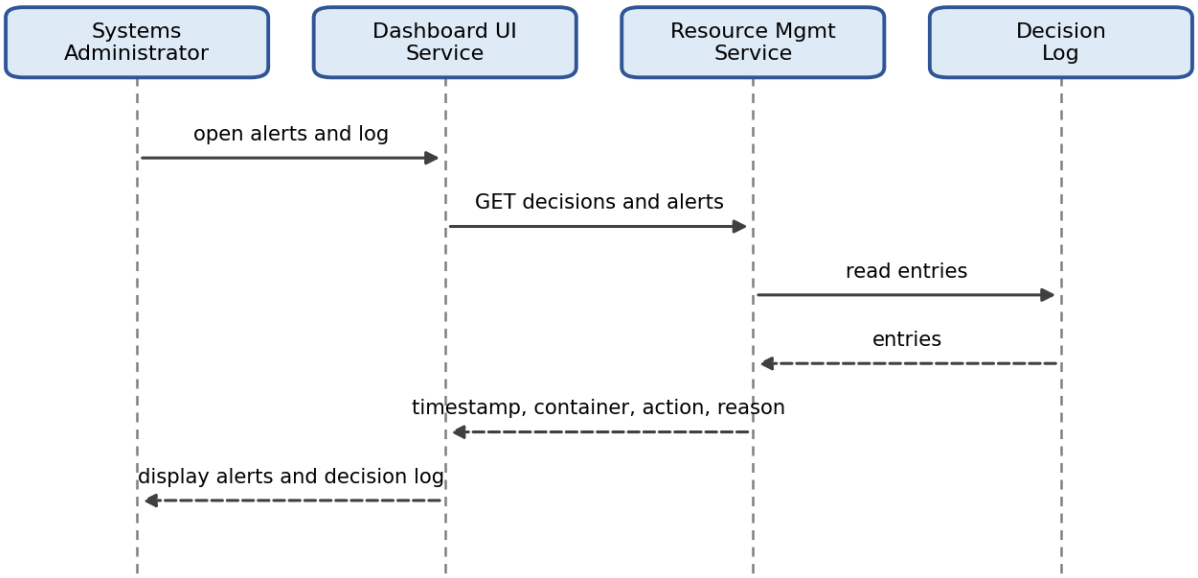


Figure 4.4. View Alerts and Decision Log sequence diagram

In the Update Resource Policy sequence (Figure 4.5), the Systems Administrator submits a policy as JSON through the REST API. The Resource Management Service validates the structure and values; if the policy is valid it is saved and becomes active immediately without a restart, and a confirmation is returned. If it is invalid, an error is returned and the previous policy remains active.

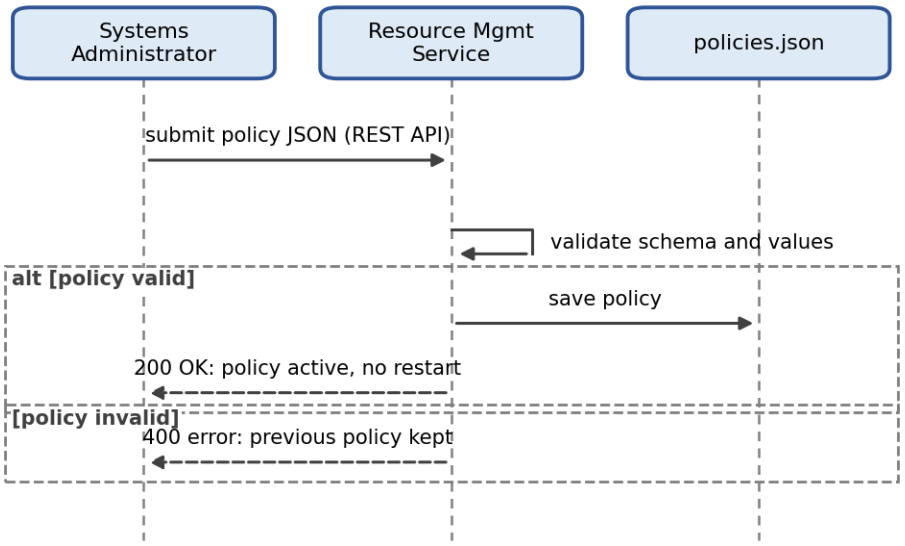


Figure 4.5. Update Resource Policy sequence diagram

Monitoring Flow

In the Collect Metrics sequence (Figure 4.6), cAdvisor reads the CPU, memory and limit statistics of the containers from the Docker Engine, Prometheus scrapes them from cAdvisor at every interval and stores them as time series, and the Monitoring Service queries Prometheus and exposes the formatted metrics to the other services through its REST and WebSocket interfaces.

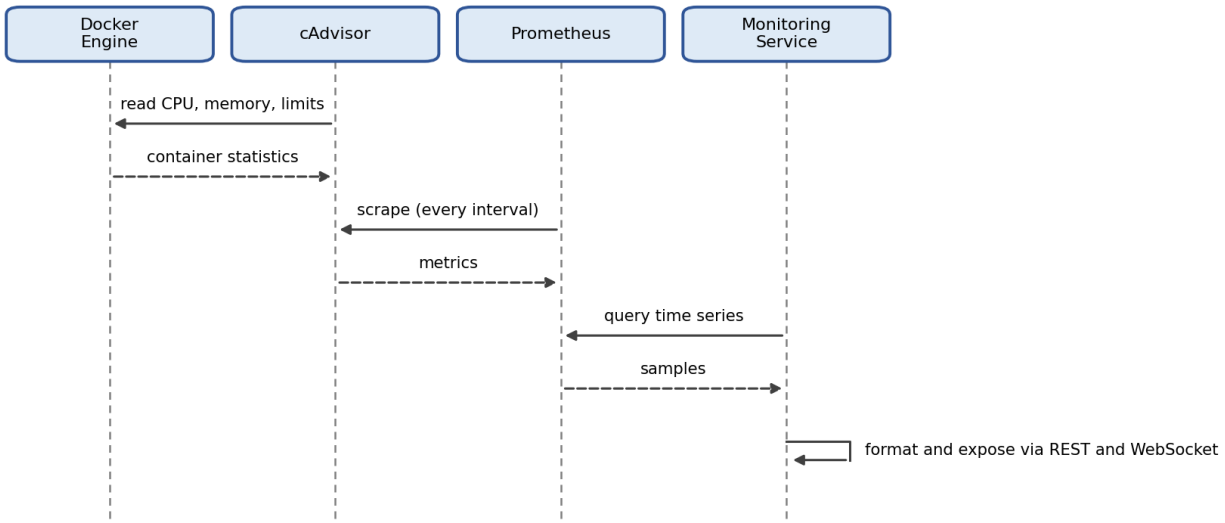


Figure 4.6. Collect Metrics sequence diagram

Control Flows

In the Evaluate Container Policy sequence (Figure 4.7), the Resource Management Service starts each cycle by checking the health of the Monitoring Service and skips the cycle if it is degraded. Otherwise it fetches the metrics of all containers, loads the policies, evaluates Rule B first and then Rule A using consecutive-breach counts and cooldown timers, and sends any resulting decision (SET_MEMORY or INCREASE_CPU) to the Orchestration Service.

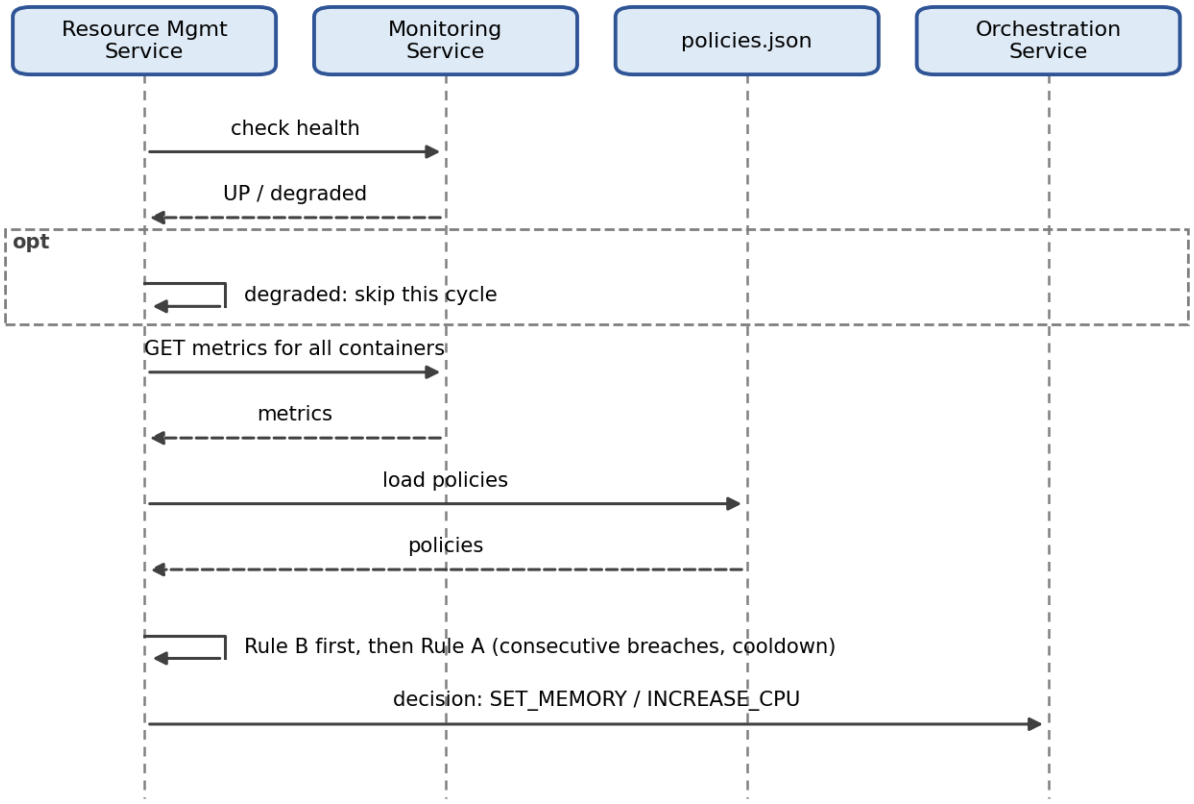


Figure 4.7. Evaluate Container Policy sequence diagram

In the Apply Host Guardrail sequence (Figure 4.8), the Resource Management Service reads the host and container memory totals from the Monitoring Service. When total container memory exceeds 98% of host RAM it freezes new scale-ups and asks the Orchestration Service to throttle the CPU of non-critical containers; while memory remains critical it asks the Orchestration Service to terminate the lowest-priority container, one at a time. When host memory falls below 90%, the freeze is lifted.

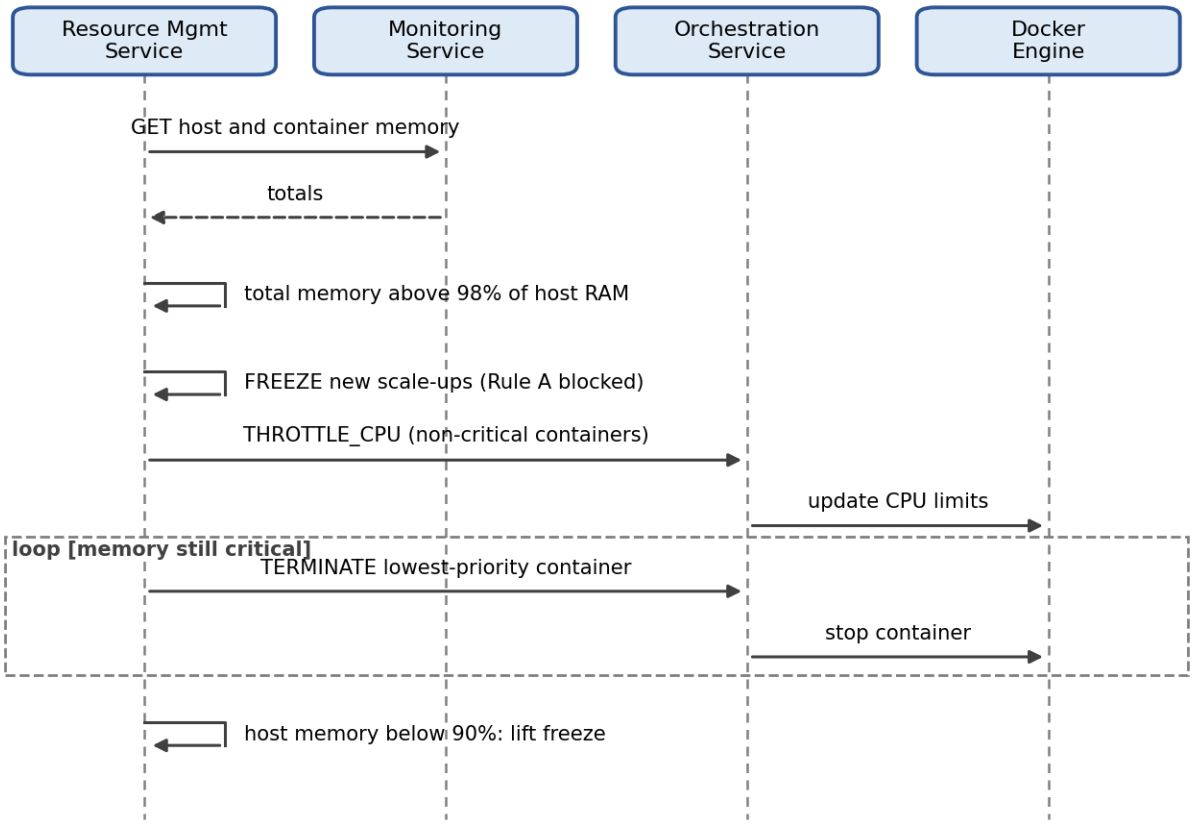


Figure 4.8. Apply Host Guardrail sequence diagram

In the Enforce Decision sequence (Figure 4.9), the Orchestration Service receives a decision from the Resource Management Service and checks that the target container exists and is not protected. It then applies the change through the Docker Engine API (updating limits or stopping the container), returns the outcome and records it in the decision log. If the Docker call fails, the failure is recorded and an alert is raised.

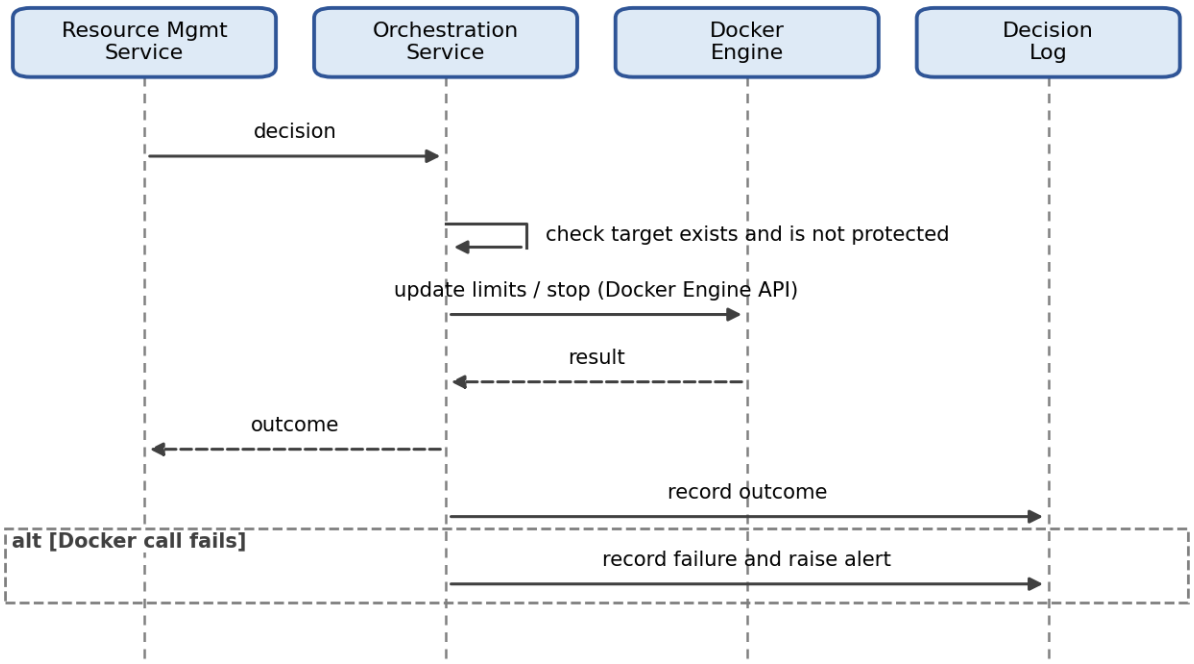


Figure 4.9. Enforce Decision sequence diagram

In the Record Decision and Raise Alert sequence (Figure 4.10), the Resource Management Service and the Orchestration Service write each decision and enforcement outcome to the decision log with a timestamp, container, action and reason. For warnings, freezes, throttling, evictions and failures, an alert is built and pushed together with the entry to the Dashboard UI Service, which displays it to the administrator.

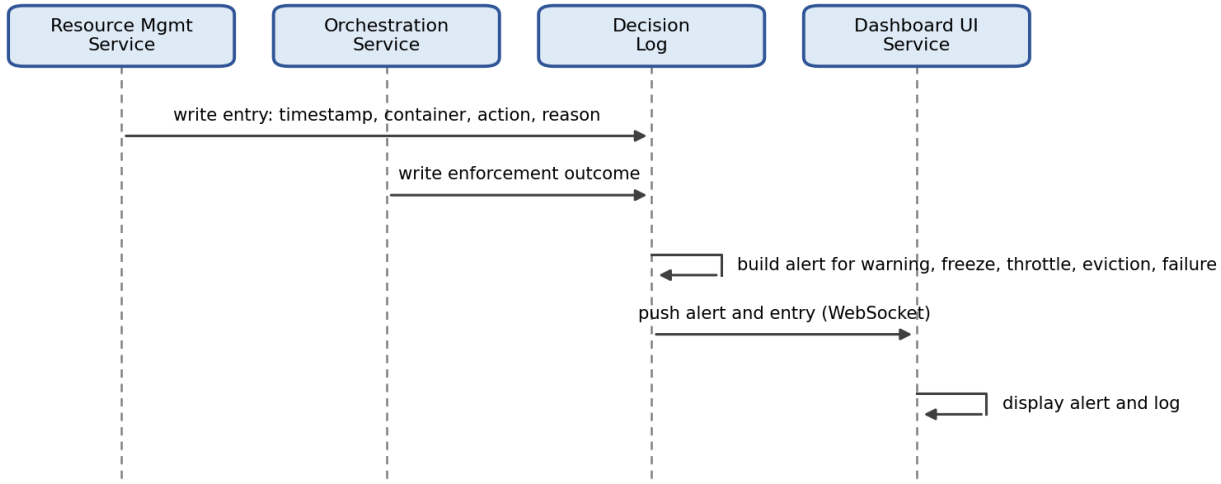


Figure 4.10. Record Decision and Raise Alert sequence diagram

5. Implementation

5.1 Methods

Implementation follows the Agile iterative approach established in Chapter 1, with each phase delivering a testable increment of the resource management system before the next begins.

Phase 1: Base Environment & Telemetry

Configures the Linux host and Docker runtime and deploys cAdvisor and Prometheus to establish live container performance telemetry. Builds the Monitoring Service, which ingests this telemetry and serves it through REST APIs and WebSockets.

Phase 2: Dynamic CPU Allocation

Implements the control loop of the Resource Management Service and the CPU allocation logic, which raises a container's CPU limit when its usage stays above the threshold for consecutive intervals and is validated against the baseline.

Phase 3: Memory Safety & Crash Prevention

Implements Rule A memory scale-up and scale-down with cooldown timers, and the Rule B host guardrail (freeze, throttle, evict), so that memory headroom is expanded before OOM failures and the host is protected from exhaustion.

Phase 4: System Integration & Benchmarking

Combines the Monitoring, Resource Management, Orchestration and Dashboard UI services into one system with run-time policy updates, and executes synthetic stress tests (CPU bursts and memory pressure) to benchmark it against static-limit baselines.

At the time of writing, the Monitoring Service and the Resource Management Service are implemented, while the Orchestration Service and the Dashboard UI Service are in development.

5.2 Technologies Used

The following specialized software tools, libraries and utilities are integrated into the engineering lifecycle:

- cAdvisor (Container Advisor, v0.49.1): Analyzes and exports real-time resource usage and performance characteristics directly from running containers.

- Prometheus (v2.51.2): Ingests, stores, and queries time-series telemetry data scraped from cAdvisor using PromQL queries.
- Docker Engine & CLI: Provides low-level containerization APIs and cgroups isolation for target container workloads.
- Docker Compose (v2.27.0): Defines and launches the telemetry stack and the workload containers from YAML configuration files.
- Dockerode (v4.0.2): Node.js client library through which the management service calls the Docker Engine API over the Unix socket to inspect containers and modify their limits at run time.
- stress-ng (v0.13.12) / sysbench (v1.0.20): Benchmarking and load-testing utilities used to generate synthetic CPU calculation bursts and rapid memory allocation requests.
- Postman / Curl: API verification tools used to inspect HTTP endpoints exposed by Prometheus and custom automation webhooks during debugging cycles.
- Git & GitHub: Version control system used to manage source code repositories, track version history, and organize work across feature branches.
- Node.js (v18 or later; v20.19.0 LTS used in testing) with Express.js (v4.19.2): Implementation platform for the backend services, used to fetch metrics, evaluate policies, expose REST APIs and WebSockets, and apply resource changes through the Docker Engine API.
- REST APIs and WebSockets: Interfaces through which the Monitoring Service serves telemetry to the other services and the dashboard, and through which policies are updated.

5.3 Implementation Environment

- Operating System: Ubuntu 22.04.4 LTS (Linux kernel 5.15, cgroup v2); development is standardized to Linux platforms to attain real-world system accuracy.
- Container Runtime: Docker Engine v26.1.1 (containerd v1.6.31, runc v1.1.12).
- Implementation Language: Node.js (v18 or later; v20.19.0 LTS used in testing) with Express.js, using the Dockerode client for the Docker Engine API.
- Version Control: GitHub to manage different versions.

6. Experimental Evaluation

6.1 Test Environment and Methodology

6.1.1 Hardware and Software Configuration

The evaluation was conducted on a dedicated single-host bare-metal testbed, configured as summarized in Table 6.1 and illustrated in Figure 6.1. The monitoring stack and the management services ran on the same host as the workload containers.

Table 6.1. Experimental testbed specifications

Layer	Component	Details / version
Host hardware	CPU	4 cores, 8 threads (AMD Ryzen 5, 2.80 GHz)
	Memory (RAM)	16[1].0 GiB DDR4
	Storage	NVMe solid-state drive (500 GB)
Operating system	Base OS	Ubuntu 22.04.4 LTS (Linux kernel 5.15.0-generic, x86_64)
	Cgroup subsystem	Cgroup v2 unified hierarchy (/sys/fs/cgroup)
Containerization	Container engine	Docker Engine Community v26.1.1, Docker Compose v2.27.0
	Container runtime	containerd v1.6.31, runc v1.1.12
Telemetry stack	Metrics exporter	Google cAdvisor v0.49.1 (privileged, host mounts)
	Time-series database	Prometheus v2.51.2 (scrape interval 1 s, retention 2 h)
Management system	Backend environment	Node.js v20.19.0 LTS, Express v4.19.2
	Control loop frequency	1.0 Hz (1-second evaluation cycle)
	Orchestration API	Dockerode v4.0.2 via Unix socket (/var/run/docker.sock)

1

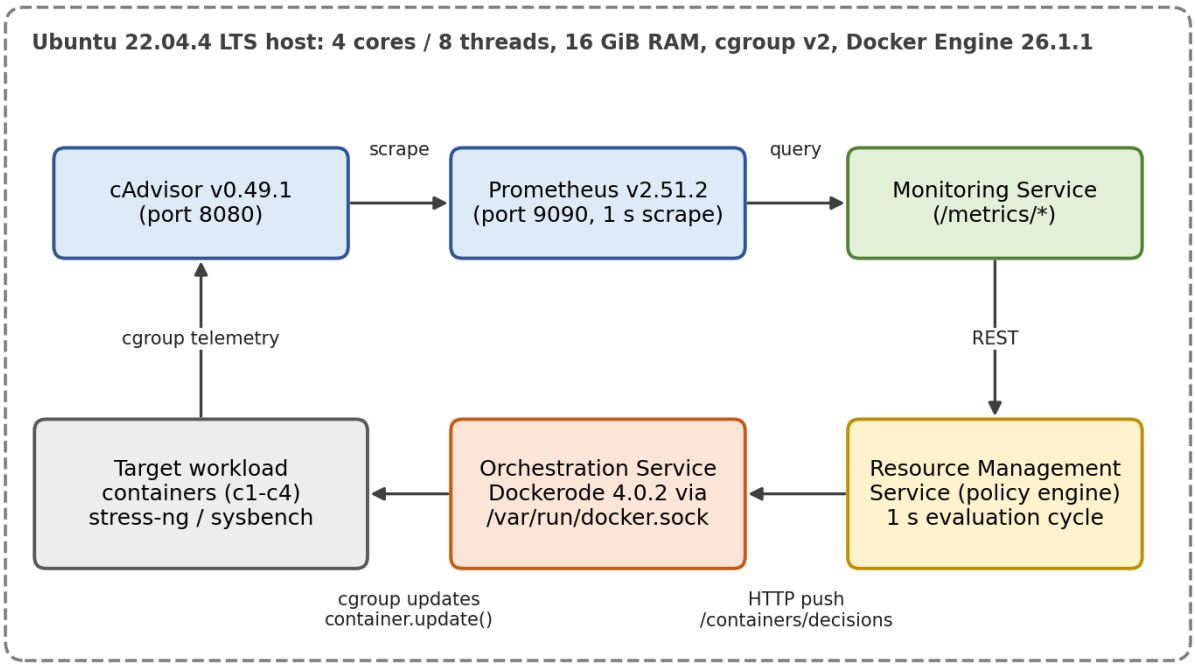


Figure 6.1. Experimental testbed

6.1.2 Benchmark Tools and Workload Profiles

Synthetic workloads were generated with stress-ng (v0.13.12) and sysbench (v1.0.20) inside Alpine Linux containers (alpine:3.19). Four scenarios were designed to evaluate the system under controlled, reproducible conditions.

- Scenario 1, normal operation (static quota versus adaptive state). The objective was to evaluate steady-state behaviour and telemetry overhead under nominal load, using a low to moderate background compute and memory footprint. Command: stress-ng --cpu 1 --cpu-load 35% --vm 1 --vm-bytes 160M --timeout 300s, in a container initialized with a 256 MiB memory limit and a CPU quota of 0.5 cores.
- Scenario 2, sustained high demand (scale-up and OOM prevention). The objective was to exercise the Rule A scale-up logic and quantify protection against out-of-memory termination (OOMKilled) and CPU throttling, using aggressive memory allocation and multi-threaded CPU stress that exceeded the initial limit. Command: stress-ng --cpu 2 --cpu-load 95% --vm 1 --vm-bytes 340M --vm-hang 0 --timeout 300s, in a container initialized with a static 256 MiB limit and 0.5 CPU cores. The 340 MiB memory demand deliberately exceeded the 256 MiB capacity to test automated scaling before kernel termination.

- Scenario 3, fluctuating spikes (hysteresis and anti-flapping). The objective was to evaluate consecutive-breach filtering, scale-down reclamation and cooldown timers under an oscillating workload that alternated between burst stress (about 95% memory for 45 seconds) and low idle load (20% memory for 45 seconds). A shell harness alternated stress-ng --vm 1 --vm-bytes 240M with sleep 45s inside an initial 256 MiB boundary.
- Scenario 4, host memory pressure (Rule B guardrail). The objective was to validate the three-stage cascade (freeze, throttle, evict) when container memory threatens host stability. Four containers ran concurrently: worker-1 and worker-2 (priority 1), batch-job (priority 0) and the critical services. Memory pressure was driven past the host guardrail threshold (hostMemoryLimitPercent of 98%; a reduced threshold of 5% was used in the validation test harness) to test freeze activation, CPU throttling of non-critical workloads, eviction of the lowest-priority batch job, and preservation of the protected services (prometheus, cadvisor, backend and dashboard).

6.1.3 Experimental Methodology

Each scenario was executed under two identical, controlled conditions, and every run was repeated five times.

- Baseline (static limits): containers ran with fixed Docker runtime allocations (--memory and --cpus), reflecting a default production environment without an active controller.
- With the management system (closed-loop control): the Monitoring Service collected metrics at 1-second intervals, the policy engine evaluated Rule A and Rule B, and the Orchestration Service applied cgroup adjustments in real time through the container update call of the Docker daemon.

The experiments used a 1-second scrape and evaluation interval, whereas the design default is 15 seconds. With the 1-second interval, three consecutive breaches correspond to about three seconds; with the default interval the same rule would take about 45 seconds.

6.2 Results

The results for the four workload scenarios are presented in Table 6.2. Values are arithmetic means across the five runs, and the figures after the ± sign show the run-to-run variation.

Table 6.2. Benchmark results

Workload scenario	Metric	Baseline (static limits)	With management system	Effect
Normal operation	CPU utilization (%)	34.8% (±0.6%)	35.1% (±0.8%)	No measurable difference (within run-to-run variation)
Normal operation	Memory utilization (%)	64.2% (±1.1%)	64.5% (±1.0%)	Stable footprint; no scaling triggered
Sustained high demand	CPU throttling (% of periods throttled)	68.4% (±2.4%)	9.2% (±1.5%)	86.5% reduction in CPU throttling
Sustained high demand	OOM kills (count)	5 of 5 runs (100%)	0 of 5 runs (0%)	OOM terminations eliminated
Fluctuating spikes	Allocation change events (count)	0 (static limits)	4 events (2 up, 2 down)	Controlled elasticity without oscillation
Host memory pressure	Time above 98% of host memory (s)	46.2 s (ended only when the kernel OOM killer intervened)	3.4 s (±0.6 s)	92.6% reduction; recovery below 90%

6.2.1 Scenario Breakdown and Timeline Analysis

Normal operation. Under nominal load both configurations ran stably. The baseline container averaged 34.8% CPU utilization and 164.3 MiB of memory (64.2% of its 256 MiB limit); with the management system active the averages were 35.1% and 64.5%. The 0.3-point CPU difference is smaller than the run-to-run variation, so no measurable effect of continuous 1-second telemetry collection on the workload was observed. Because utilization stayed below the 90% scale-up threshold and above the 35% scale-down threshold, no allocation changes were triggered.

Sustained high demand. The workload tried to allocate 340 MiB of RAM and to saturate two CPU cores. In the baseline, with static limits of 256 MiB and 0.5 cores, 68.4% of CFS periods were throttled, and when memory allocation reached 255.8 MiB the Linux OOM killer terminated the container (OOMKilled: true, exit code 137) within 18 seconds of startup in all five runs. With the management system, the Monitoring Service reported the surge within 1.0 second of the container crossing 90% utilization (230.4 MiB). After the third consecutive breach, at t = 3 s, the Resource Management Service emitted a SET_MEMORY decision targeting 512 MiB (a 256 MiB step), and the Orchestration Service applied it through the Docker update operation in 38 ms without restarting the process. The workload completed its 340 MiB allocation without an OOM event in any run. In parallel, Rule A detected CPU demand above 90% and raised the CPU allocation from 0.5 to 1.0 cores, which reduced throttling from 68.4% to 9.2%. Figure 6.2 shows the sequence of events.

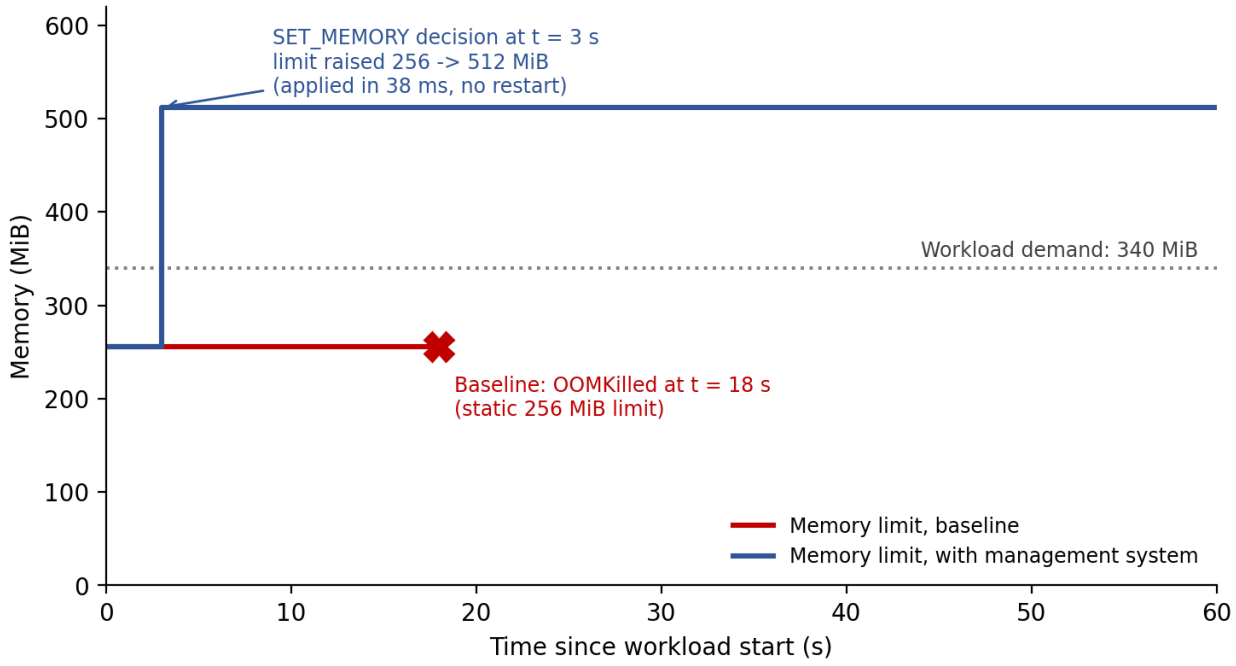


Figure 6.2. Sustained high demand: memory limit with and without the management system (schematic of logged events)

Fluctuating spikes. The workload alternated 45-second phases of high demand (about 95% of the initial memory limit) and idle decay (20% memory). When memory crossed the 90% threshold for three consecutive intervals at t = 3 s, the engine emitted SET_MEMORY (256 MiB to 512 MiB) and started a 60-second cooldown (cooldownSeconds of 60). Short spikes during the

cooldown were dampened, which prevented rapid multi-step over-allocation. When the workload went idle, memory fell below 35% (179 MiB of 512 MiB), and after eight consecutive idle intervals (scaleDownConsecutiveIntervals of 8) the engine began the scale-down. It first checked the safety floor, max(minMiB, usage × (1 + headroom)) = max(128, 102 × 1.25) = 128 MiB, and then reduced the limit from 512 MiB to 256 MiB, the container's initial allocation. Over the 10-minute test exactly four allocation events occurred (two up and two down), and no flapping was observed.

Host memory pressure. Total container memory was driven past the host guardrail threshold. In the baseline, aggregate container allocation crossed 98% of host memory, the host became unresponsive through memory starvation and swap thrashing, and critical pressure persisted for 46.2 seconds on average until the kernel OOM killer terminated processes indiscriminately, including critical monitoring components. With the management system, host memory exceeded 98% for two consecutive cycles (t = 2 s) and Rule B entered Stage 1 (Freeze), broadcasting FREEZE so that all pending and new scale-up requests were blocked. At t = 3 s it entered Stage 2 (Throttle), reducing all non-protected workloads (worker-1, worker-2 and batch-job) to the baseline CPU quota of 0.25 cores to slow memory churn. At t = 4 s (terminateAfterBreaches of 4) it entered Stage 3 (Evict), selecting the lowest-priority container (batch-job, priority 0) and issuing a TERMINATE decision; the container was stopped and removed in 410 ms. Host memory then fell below the 90% recovery boundary, the experiment log recording a time above the 98% threshold of 3.4 seconds on average, and an UNFREEZE decision was logged to restore normal operation (host memory was 82.4% at t = 6 s). Throughout, all protected containers (prometheus, cadvisor, backend and dashboard) ran uninterrupted. Figure 6.3 shows the sequence of events.

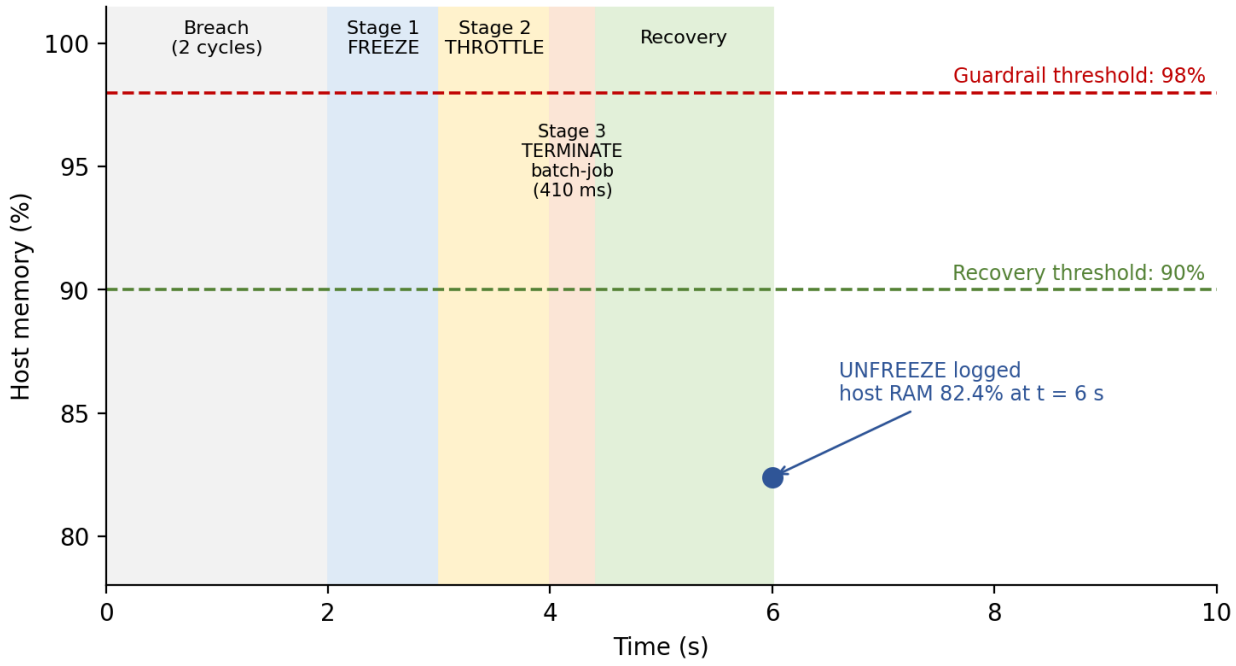


Figure 6.3. Host memory pressure: Rule B stages and recovery (schematic of logged events)

6.3 Performance Analysis

6.3.1 Comparative Assessment: Static Baseline versus Autonomous Management

The results show a clear advantage of closed-loop resource management over static limits for the workloads tested.

- Container survival: with static limits, the memory limit is a rigid ceiling and the kernel terminates the container when demand exceeds it. The management system removed all OOM terminations (0 of 5 runs against 5 of 5) by expanding the memory limit ahead of kernel intervention, for a workload whose demand exceeded the initial limit but stayed within the configured maximum.
- CPU throttling: static quotas under heavy multi-threaded demand caused severe CFS throttling (68.4% of periods). By detecting sustained saturation and raising the CPU allocation in 0.25-core steps (up to the configured maximum of 4 cores; 0.5 to 1.0 cores in the tested run), throttling fell to 9.2%.
- Reclamation: in the fluctuating scenario the limit was returned to 256 MiB after the idle phase, so the extra memory was not held once demand had fallen.

- Host protection: Rule B kept the host recoverable and the protected services running, whereas the unmanaged host locked up until the kernel OOM killer intervened.

6.3.2 Evaluation Against Non-Functional Requirements

Table 6.3 compares the measured behaviour with the non-functional requirements of Section 3.2. Requirements that the experiments did not exercise are marked accordingly.

Table 6.3. Non-functional requirement compliance

Requirement	Target	Measured	Status
Monitoring overhead	No more than 5% of host CPU on average	0.8% to 1.4% CPU (1.1% of a single core on average); 42.6 MiB of memory	Met
Near real-time refresh	Refresh interval of 15 s or less	1.0 s scrape interval; 12 ms REST probe latency	Met
Concurrent containers	At least 20 containers	Policy evaluation across 10 containers took about 3.8 ms; end-to-end tests used four workload containers	Partially verified
Runtime stability	No restarts or OOM kills caused by Rule A adjustments	0 restarts and 0 OOM kills; limits updated live	Met
Boundary enforcement	No allocation beyond predefined maxima	Allocations capped at maxMiB of 2048 and maxCores of 4; an alert was raised at the maximum	Met
Recovery from component failure	Recovery within 60 s without manual intervention	Cycles skipped while Prometheus was down; evaluation resumed on the next interval after restart	Met
Host guardrail reaction	Host memory below 90% within 15 s of activation	3.4 s	Met

Minimal monitoring overhead. The monitoring subsystem (cAdvisor, Prometheus and the Express REST and WebSocket gateway) ran throughout all tests. Measurements with top and

docker stats showed that the telemetry and decision loop together used 1.1% of a single CPU core and 42.6 MiB of resident memory on average, so high-frequency telemetry did not noticeably disturb the workloads.

Near real-time telemetry and low-latency decisions. The control cycle was responsive at the sub-second level for each stage:

- Prometheus ingestion of cAdvisor metrics: about 850 ms.
- Response of the /metrics/memory and /metrics/cpu endpoints: about 14 ms.
- Policy engine evaluation across 10 containers: about 3.8 ms.
- Orchestration enforcement round trip (Docker update): 35 to 45 ms.

The total time from a container crossing the 90% memory threshold to its cgroup limit being expanded was under 3.2 seconds, dominated by the three consecutive 1-second samples required by the rule. This makes the scrape interval the main tuning parameter: with the default 15-second interval the same rule would take about 45 seconds.

Stability during reallocation. Limits were changed through Docker's cgroup v2 controls (memory.max and cpu.max) via Dockerode, without pausing, terminating or restarting the target container. A continuous TCP ping stream to an HTTP server inside the container showed zero packet loss and zero dropped connections during memory expansion.

Safety guardrails, anti-flapping and boundary enforcement. The policy engine respected its configured limits:

- Scale-up requests were bounded by maxMiB of 2048 and maxCores of 4. When a container stayed under unrelenting pressure at maximum capacity, the engine raised an alert (at maximum memory quota) without exceeding the envelope.
- Scale-down was protected by a 25% safety headroom, which prevented premature contraction that would immediately trigger another expansion.
- The 60-second cooldown ensured that brief, intermittent surges did not destabilize the allocation state.

Resilience to monitoring failures. To test fault tolerance, the Prometheus container was stopped (docker stop prometheus) during active evaluation. The Monitoring Service immediately reported a DEGRADED status (HTTP 503) on its health endpoint, and the Resource Management control loop logged "[RM] Monitoring service degraded, skipping cycle" and

withheld all scaling operations, which prevented decisions based on stale data. When Prometheus was restarted, the health probe returned UP on the next interval and autonomous evaluation resumed without manual intervention.

Multi-tier host guardrail. The cascade in Rule B behaved as designed:

- Tier 1 (Freeze) stopped existing containers from scaling up and so stopped aggregate host memory from growing further.
- Tier 2 (Throttle) capped non-critical workloads at 0.25 CPU cores, which slowed heap allocation in memory-stressed applications.
- Tier 3 (Prioritized eviction) selected the lowest-priority container (batch-job, priority 0) instead of the higher-priority workers (worker-1 and worker-2, priority 1).
- Protected isolation: the system containers (prometheus, cadvisor, backend and dashboard) were never throttled or evicted, which preserved observability and control during the critical period.

7. Discussion

7.1 Strengths

The system's principal strength is its closed-loop design: cAdvisor and Prometheus feed continuously updated metrics into a control loop that evaluates policies, decides and enforces limit changes while containers keep running. Separating deciding (Resource Management Service) from enforcing (Orchestration Service) keeps the policy logic testable without touching Docker. Policies are held as validated data and can be changed at run time through a REST API without restarting services, and every decision is logged with its reason, which makes the system auditable. Safety is built in: consecutive-breach detection, cooldown timers and a wide gap between the scale-up and scale-down thresholds prevent flapping, and the host-wide guardrail protects the machine even when per-container rules would demand more resources. The approach adjusts the limits of individual running containers directly, whereas the HPA only changes replica counts [16]. The evaluation supports these strengths: OOM kills fell from 5 of 5 runs to 0 of 5, CPU throttling fell from 68.4% to 9.2%, and the host guardrail kept the protected services running. Finally, the system is built from proven open-source components (cAdvisor,

Prometheus, Docker, Node.js), and because it needs only a single Docker host it is simple to deploy and requires no cluster.

7.2 Weaknesses and Limitations

Threshold-based control remains fundamentally reactive: like the default autoscalers of orchestrators such as Kubernetes [16], it acts only after utilization crosses a configured boundary. Requiring several consecutive breaches avoids overreacting to short spikes but delays the response: in the evaluation, with a 1-second interval, a limit was raised about 3 seconds after the breach, whereas with the default 15-second interval the same rule would take about 45 seconds. Predictive systems such as Theta-Scan [19] anticipate demand, and event-driven systems such as Escra [20] react to near-OOM kernel events within sub-second timeframes. The evaluation also has limits: it ran on a single Docker host with four workload containers, compared the system only with static limits rather than with the HPA or VPA, and used synthetic stress-ng and sysbench workloads, and the 20-container target was not exercised. In the host-pressure scenario the baseline run ended only when the kernel OOM killer intervened, so its 46.2 seconds is a lower bound rather than a like-for-like duration. Without an orchestrator the system provides no scheduling, replica scaling or self-healing of its own. The last stage of the host guardrail terminates containers, so its outcome depends on workloads being given sensible priorities and on critical containers being protected, and because the dashboard is read-only, administrators can influence behaviour only by changing the policies. Consistent with the scope exclusions, the system does not manage network bandwidth or disk I/O, and it targets only containerized workloads rather than virtual machines.

7.3 Comparison with Existing Work

Relative to the HPA and VPA, whose reactive adjustments follow already-breached thresholds and can produce scaling delays, SLA violations, and severe throttling during spikes [16], this project's threshold evaluation, which scales a limit up once usage stays above 90% of it, raised the limit before the kernel limit was reached in all five sustained-demand runs; Chapter 6 compared the system only with static limits, so whether it responds sooner than the HPA or VPA remains to be tested. Theta-Scan represents a stronger predictive alternative, using behavior-driven forecasting to adjust limits before exhaustion events occur [19]; this project adopts the simpler threshold design and defers predictive modeling to future work. Escra's event-driven, sub-second allocation [20] reacts faster than periodic threshold polling, but requires kernel event

instrumentation beyond this project's scope. Finally, published evaluations report performance degradation under contention even when an orchestrator's resource management is in place [6]; the present system addresses this at the level of a single Docker host by adjusting limits directly, although it does not replace the scheduling and recovery functions that orchestrators provide.

8. Conclusions and Future Work

This project set out to design and develop a Container-Based Resource Management System that monitors container CPU and memory usage in real time with cAdvisor and Prometheus and automatically manages resource limits on Docker through the Docker Engine API. The design combines four cooperating microservices (monitoring, resource management, orchestration and a read-only dashboard UI), a closed control loop (measure, evaluate policies, decide, enforce, measure again), per-container threshold rules with a host-wide guardrail, and policies held as run-time updatable data, and is specified through a use case model and sequence diagrams covering administrator, monitoring and control flows. Implementation followed four Agile phases, from base environment and telemetry through dynamic CPU allocation and memory safety to integration and benchmarking. Evaluated against static limits on a single-host testbed with stress-generated workloads, the system eliminated OOM terminations (0 of 5 runs, against 5 of 5 for static limits), cut CPU throttling from 68.4% to 9.2%, kept allocation changes to four stable events without flapping under fluctuating load, and brought host memory back under control in 3.4 seconds while the protected services kept running, at a monitoring cost of about 1.1% of one CPU core and 42.6 MiB of memory.

Future work includes comparing the controller directly with the Kubernetes HPA and VPA, evaluating it with 20 or more containers and on multiple hosts, extending the threshold controller with predictive or machine-learning-based allocation in the direction of systems such as Theta-Scan [19], and integrating with an orchestrator such as Kubernetes so that the custom control layer can complement the platform's native autoscalers.

References

[1] R. Queiroz, T. Cruz, J. Mendes, P. Sousa, and P. Simoes, "Container-based Virtualization for Real-time Industrial Systems - A Systematic Review," ACM Comput. Surv., vol. 56, no. 3, pp. 1-38, Mar. 2024, doi: 10.1145/3617591.

[2] B. B. Rad, H. J. Bhatti, and M. Ahmadi, "An Introduction to Docker and Analysis of its Performance," 2017.

[3] K. Senjab, S. Abbas, N. Ahmed, and A. U. R. Khan, "A survey of Kubernetes scheduling algorithms," J. Cloud Comput., vol. 12, no. 1, p. 87, Jun. 2023, doi: 10.1186/s13677-023-00471-1.

[4] S. Singh, F. Shivanshi, and R. Jain, "A Framework for Evaluating Container Performance Across Diverse Kubernetes Environments," in Proc. 2024 IEEE Int. Conf. Cloud Computing, Engineering & Tech., 2024, pp. 12-17, doi: 10.1109/GKE-EKS.10815769.

[5] G. Lahmann, T. McCann, and W. Lloyd, "Container Memory Allocation Discrepancies: An Investigation on Memory Utilization Gaps for Container-Based Application Deployments," in Proc. 2018 IEEE Int. Conf. Cloud Eng. (IC2E), Orlando, FL, USA, 2018, pp. 404-405, doi: 10.1109/IC2E.2018.00076.

[6] E. Kim, K. Lee, and C. Yoo, "On the Resource Management of Kubernetes," in Proc. 35th International Conference on Information Networking (ICOIN), Jeju Island, South Korea, 2021, pp. 154-158, doi: 10.1109/ICOIN50884.2021.9333977.

[7] P. Liu and J. Guitart, "Performance comparison of multi-container deployment schemes for HPC workloads: an empirical study," The Journal of Supercomputing, vol. 77, pp. 6273-6312, 2020, doi: 10.1007/s11227-020-03518-1.

[8] C. Avasalcai, B. Zarrin, P. Pop, and S. Dustdar, "Efficient Hosting of Robust IoT Applications on Edge Computing Platform," in 2020 IEEE 4th International Conference on Fog and Edge Computing (ICFEC), pp. 1-10, 2020, doi: 10.1109/icfec50348.2020.00008.

[9] B. Barua and M. S. Kaiser, "AI-Driven Resource Allocation Framework for Microservices in Hybrid Cloud Platforms," arXiv, 2024, doi: 10.48550/arxiv.2412.02610.

[10] S. Agarwala, C. Poellabauer, J. Kong, K. Schwan, and M. Wolf, "System-Level Resource Monitoring in High-Performance Computing Environments," J. Grid Comput., vol. 1, no. 3, pp. 273-289, Sep. 2003, doi: 10.1023/B:GRID.0000035189.80518.5d.

[11] G. Juve et al., "Practical Resource Monitoring for Robust High Throughput Computing," in 2015 IEEE International Conference on Cluster Computing, Chicago, IL: IEEE, Sep. 2015, pp. 650-657, doi: 10.1109/CLUSTER.2015.115.

[12] M. Andreolini, M. Colajanni, M. Pietri, and S. Tosi, "Real-time adaptive algorithm for resource monitoring," in Proceedings of the 9th International Conference on Network and Service Management (CNSM 2013), Zurich, Switzerland: IEEE, Oct. 2013, pp. 67-74, doi: 10.1109/CNSM.2013.6727811.

[13] T. DeWitt, T. Gross, B. Lowekamp, N. Miller, P. Steenkiste, J. Subhlok, and D. Sutherland, "ReMoS: A Resource Monitoring System for Network-Aware Applications," Tech. Rep. CMU-CS-97-194, Carnegie Mellon University, Pittsburgh, PA, USA, Dec. 1997, doi: 10.21236/ADA352304.

[14] C.-C. Chang, S.-R. Yang, E.-H. Yeh, P. Lin, and J.-Y. Jeng, "A Kubernetes-Based Monitoring Platform for Dynamic Cloud Resource Provisioning," in GLOBECOM 2017 - 2017 IEEE Global Communications Conference, Singapore: IEEE, Dec. 2017, pp. 1-6, doi: 10.1109/GLOCOM.2017.8254046.

[15] C. Batey, "CPU Quota for Docker and Kubernetes," Christopher Batey Blog, May 2020. [Online]. Available: https://batey.info/cgroup-cpu-quota-for-docker.html.

[16] R. Soth, P. Vann, and L. Sok, "Kubernetes Resource Optimization Using Machine Learning: A Comprehensive Review of Widely Used Techniques," Preprints 2026, 2026061718, doi: 10.20944/preprints202606.1718.v1.

[17] A. Mustyala, "Dynamic Resource Allocation in Kubernetes: Optimizing Cost and Performance," EPH-International Journal of Science and Engineering, vol. 7, no. 3, pp. 59-71, Sep. 2021. [Online]. Available: http://ephijse.com/index.php/SE/article/view/237.

[18] D. Medeiros et al., "Vertical Resource Adaptivity for HPC in Containerized Environments," arXiv preprint, 2025. [Online]. Available: https://www.alphaxiv.org/pdf/2505.02964v1.

[19] J. L. Berral, D. Buchaca, C. Herron, C. Wang, and A. Youssef, "Theta-Scan: Leveraging Behavior-Driven Forecasting for Vertical Auto-Scaling in Container Cloud," in 2021 IEEE 14th International Conference on Cloud Computing (CLOUD), Chicago, IL, USA, 2021, pp. 404-409, doi: 10.1109/CLOUD53861.2021.00054.

[20] G. Cusack et al., "Escra: Event-driven, Sub-second Container Resource Allocation," in 2022 IEEE 42nd International Conference on Distributed Computing Systems (ICDCS), Bologna, Italy, 2022, pp. 313-324, doi: 10.1109/ICDCS54860.2022.00038.

[21] D. Merkel, "Docker: Lightweight Linux Containers for Consistent Development and Deployment," Linux Journal, vol. 2014, no. 239, pp. 2-8, May 2014.

[22] D. Beserra, E. D. Moreno, P. T. Endo, J. Barreto, S. Fernandes, and D. Sadok, "Performance analysis of Linux containers for high performance computing applications," International Journal of Grid and Utility Computing, vol. 8, no. 4, pp. 321-329, 2017.